

NoboGhat

Software Requirements Specification (SRS) - Part 2

Functional Requirements • Non-functional Requirements • User Personas • User Stories • System Architecture

1. Functional Requirements

ID	Requirement	Description
FR-01	User Registration	Users can create accounts as Farmer, Trader or Boat Owner.
FR-02	Authentication	Secure login/logout with role-based access.
FR-03	Boat Listing	Boat owners can register and manage boats.
FR-04	Route Management	Owners define routes, schedules and capacity.
FR-05	Cargo Booking	Users book cargo space instead of entire boats.
FR-06	Capacity Validation	Prevent bookings exceeding remaining capacity.
FR-07	Booking History	Users view and cancel eligible bookings.
FR-08	Admin Dashboard	Monitor users, boats, bookings and reports.
FR-09	Search	Search by route, district and departure date.
FR-10	Notifications	Booking confirmation and status updates (future-ready).

2. Non-functional Requirements

- Usability: First-time users should complete a booking in under 5 minutes.
- Performance: Common pages should respond within 2 seconds under normal load.
- Security: Password hashing, authenticated access, and input validation.
- Availability: Accessible from desktop and mobile browsers.
- Scalability: Architecture should support migration from JSON storage to MySQL/PostgreSQL.
- Maintainability: Modular code with clear separation of frontend, backend, and services.
- Reliability: Booking validation must prevent overbooking.
- Localization: Designed to support Bangla and English interfaces.

3. User Personas

Farmer

Needs affordable transport for small harvests. Goal: share cargo space and reduce costs.

Small Trader

Ships products between districts. Goal: reliable scheduling and transparent pricing.

Boat Owner

Wants higher boat utilization and easier booking management.

Administrator

Maintains system integrity, verifies operators and resolves issues.

4. User Stories

- As a Farmer, I want to book only 100 kg of cargo so I don't have to hire an entire boat.
- As a Boat Owner, I want to publish available trips so customers can find me online.
- As a Trader, I want to compare routes before booking.
- As an Administrator, I want to approve boat registrations to reduce fraud.
- As a User, I want to see my booking history for future reference.

5. High-Level System Architecture

User Browser



Frontend (HTML • CSS • JavaScript)

↓ HTTPS / REST API

Backend (Java Spring Boot planned)



Business Logic (Booking, Routes, Capacity Validation)



Database (MySQL planned; JSON for MVP)



Future Integrations: Weather API • SMS • Digital Payments • GPS

Design Philosophy: The architecture is intentionally layered so that the frontend, business logic, and data layer remain independent. This makes the system easier to test, maintain, and expand as NoboGhat evolves from an academic project into a production-ready platform.